Exception Handling in Python

1. Introduction to Exceptions

What is an Exception?

- An **exception** is an **unexpected event** that occurs during the execution of a program.
- It disrupts the normal flow of the program's instructions.
- In simpler terms, it is a **runtime error** that Python detects and stops the program for.

What is Exception Handling?

- The process of responding to the occurrence of exceptions to prevent the program from crashing abruptly.
- It allows the program to complete its execution or terminate gracefully by showing a user-friendly error message.

---

2. Types of Errors in Python

Errors are broadly classified into **3 categories** based on when they occur and how they affect the program.

🔹 1. Syntax Errors (Compile-Time Errors)

- **Definition:** Violations of Python's grammar rules. Detected by the interpreter **before** the program executes.
- **Program Status:** The program **does NOT run** at all.
- **Key Feature:** Caught before execution. Program does not run.

Common Syntax Errors

Error Type	Example (Incorrect)	Correction
Missing Colon `:`	`if x > 5` `print("Hello")`	Add `:` after `if x > 5`
Missing Parentheses	`print("Hello"`	Add closing `)`
Missing Quotes	`print(Hello)`	Use `"Hello"`

Indentation Error	`if x > 5:`
	`print("Hi")`
	`print("Bye")`
		Indent `print("Bye")` correctly

Invalid Variable Name	`123var = 10`	Use `var123 = 10`
Missing Operator	`c = a b`	Use `c = a * b`
Keyword as Identifier	`class = "XII"`	Use `cls = "XII"`
Unclosed String	`name = 'John`	Add closing quote `'John'`

🔹 2. Run-time Errors (Exceptions)

- **Definition:** Occur **during execution**. Syntax is correct, but the program encounters an unhandled situation.
- These are the **exceptions** we handle using `try-except`.
- If not handled, the program **crashes abruptly**.
- **Key Feature:** Detected during execution. Can be handled with `try-except`.

Common Built-in Exceptions (Know for MCQs)

Exception	Example Scenario	Cause
`ZeroDivisionError`	`result = 10 / 0`	Division by zero
`NameError`	`print(age)` (age undefined)	Using undefined variable
`TypeError`	`"Hello" + 5`	Incompatible data types
`ValueError`	`int("abc")`	Correct type, invalid value
`IndexError`	`lst[5]` for `lst=[1,2,3]`	Index out of range
`KeyError`	`d["B"]` when `d={"A":1}`	Missing dictionary key

Exception	Example Scenario	Cause
`FileNotFoundError`	`open("missing.txt","r")`	File not present
`IOError`	Input/Output operation fails	General I/O failure
`AttributeError`	`lst.appendd(3)`	Calling non-existent method
`OverflowError`	`math.exp(10000)`	Result too large
`KeyboardInterrupt`	User presses `Ctrl+C`	Manual interruption

🔹 3. Logical Errors (Semantic Errors)

- **Definition:** Errors in the program's **logic**. The program runs completely but produces **incorrect output**.
- No error message is displayed. **Hardest** to detect and fix.
- **Key Feature:** Program runs, gives wrong output. No error message. Requires debugging.

Common Logical Errors

Error Type	Example (Incorrect Logic)	Expected vs Actual
Incorrect Formula	`avg = sum / 2` (should be `sum/count`)	Divides by 2 instead of 5
Wrong Condition	`if marks >= 33: print("Fail")`	Condition reversed; should be `< 33`
Off-by-One	`range(1,10)` (intent: loop 10 times)	9 iterations, should be `range(1,11)`
Wrong Operator	`area = 2 * 3.14 * radius` (intent: area)	Formula for circumference, not area
Incorrect Variable Update	`total = i` inside loop (intent: sum)	Reassigns instead of adding; use `total += i`
String vs Integer	`age = input(); if age > 18:`	String comparison; convert to `int(age)`

📊 Comparison Table (Very Important for Exams)

Feature	Syntax Errors	Run-time Errors	Logical Errors
When Detected	Before execution (Compile time)	During execution (Run time)	During testing (After execution)
Does program run?	No, stops immediately	Yes, but may crash if unhandled	Yes, runs completely
Error Message?	Yes, with line number	Yes, exception type + traceback	No error message
Can it be handled?	No, must fix code	Yes, using `try-except`	No, must fix logic
Difficulty to Fix	Easy	Medium	Hardest

---

3. Handling Exceptions: `try-except-finally`

Syntax Structure

```
try:
    # Block of code where an exception might occur
    # (Risky code)
except ExceptionType1:
    # Code to execute if ExceptionType1 occurs
except ExceptionType2:
    # Code to execute if ExceptionType2 occurs
else:
    # Code to execute if NO exception occurs in the 'try' block
finally:
    # Code that will ALWAYS execute, regardless of whether an exception occurred or not.
    # (Used for cleanup operations like closing files)
```

A. The `try` Block

- It is the **mandatory** starting block.
- Contains the code that might raise an exception.
- If no exception occurs, Python executes the `try` block completely and skips all `except` blocks.

B. The `except` Block

- Handles the exception if it occurs.
- You can have **multiple** `except` blocks for different error types.
- **Generic** `except` **:** Catches any exception (use with caution, usually as a fallback).

Example 1: Specific Exception

```
try:
    num = int(input("Enter a number: "))
    result = 100 / num
    print("Result is:", result)
except ZeroDivisionError:
    print("Error: You cannot divide by zero!")
```

Example 2: Multiple Specific Exceptions

```
try:
    num = int(input("Enter a number: "))
    result = 100 / num
    print("Result is:", result)
except ZeroDivisionError:
    print("Cannot divide by zero.")
except ValueError:
    print("Invalid input! Please enter a numeric value.")
```

Example 3: Single `except` for Multiple (tuple)

```
try:
    num = int(input("Enter a number: "))
    result = 100 / num
    print("Result is:", result)
except (ZeroDivisionError, ValueError) as e:
    print(f"An error occurred: {e}")
```

Example 4: Generic `except` (catches all)

```
try:
    num = int(input("Enter number: "))
    print(10 / num)
except:  # Blank except catches EVERYTHING (Bad practice, but works)
    print("Something went wrong")
```

📌 **Exam Tip:** The order of `except` blocks matters. Put **specific exceptions first**, and generic ones last.

C. The `else` Block (Optional)

- Executes **only if** the `try` block did **not** raise any exception.
- Useful for code that should run only when the risky part is successful.

```
try:
    num = int(input("Enter a number: "))
    result = 100 / num
except ZeroDivisionError:
    print("Cannot divide by zero.")
else:
    print(f"Division successful! Result is: {result}")
    print("Thank you for entering a valid number.")
```

D. The `finally` Block (Optional but Important)

- **Always executes**, irrespective of whether an exception occurred or not.
- Even executes if an exception is raised but not caught, or if the program has a `return` statement.
- **Use Case:** Cleanup actions – closing files, database connections, releasing resources.

```
try:
    file = open("data.txt", "r")
    content = file.read()
    print(content)
except FileNotFoundError:
    print("File not found.")
finally:
    print("Closing the file...")
    try:
        file.close()
    except:
        pass  # In case file was never opened
```

---

4. Complete Program (All Blocks + All Error Types)

```
print("===== COMPLETE EXCEPTION HANDLING DEMO =====\n")

# 1. SYNTAX ERROR (Uncomment to see - program won't run)
# print("Hello World"   # Missing closing parenthesis

# 2. RUNTIME ERROR HANDLING (try-except-else-finally)
try:
    age = int(input("Enter your age: "))
    if age < 0:
        raise ValueError("Age cannot be negative")
    days = age * 365
    print(f"You are approximately {days} days old.")
```

```
except ValueError as e:
    print(f"Runtime Error (ValueError): {e}")
except OverflowError:
    print("Runtime Error: Number is too large.")
else:
    print("Age calculation completed successfully.")
finally:
    print("--- End of age calculation program. ---\n")

# 3. LOGICAL ERROR (runs but gives wrong output)
marks1, marks2, marks3 = 85, 90, 78
total = marks1 + marks2          # INCORRECT: forgot marks3
print(f"Wrong Total (Logical Error): {total}")   # 175 (should be 253)
total_correct = marks1 + marks2 + marks3
print(f"Correct Total: {total_correct}")         # 253
```

📤 Output (if user enters 20)

```
==== COMPLETE EXCEPTION HANDLING DEMO ====

Enter your age: 20
You are approximately 7300 days old.
Age calculation completed successfully.
--- End of age calculation program. ---

Wrong Total (Logical Error): 175
Correct Total: 253
```

---

5. Key Points for CBSE Board Exams

- `try` **is mandatory** when using `except`, `else`, or `finally`.
- You **cannot** have `except` without `try`.
- `finally` is often used for **cleanup actions** (e.g., `file.close()`).

- If an exception occurs and is **not handled**, the program **terminates abruptly**.
- Python checks exceptions in the order they are written. Put the **most specific exceptions first**.
- `raise` keyword: Used to manually trigger an exception.
- **Syntax Error vs Runtime Error:** Syntax errors are caught before execution; runtime errors (exceptions) are caught during execution and can be handled.

📌 Quick Revision Flowchart


